DevMatch AI

Complete project explanation, DBMS viva preparation, presentation script, and interview questions

What this is	Verified project state
Application	Flask recruitment and developer portfolio website with MySQL as the application database.
Users	Developer and recruiter accounts, job listings, applications, profile skills, resumes, and an optional Gemini advisor.
Local check	On 29 Sep 2026, the DBMS verifier passed: 10 application tables, each with at least five rows and five attributes. An additional status-history table is present.
Important scope	Supabase is optional private PDF storage. It is not the application database. The local website uses MySQL and local resume files.

How to use this guide: Read sections 1-5 to understand the system, 6-12 to explain implementation, 13-16 to rehearse the demonstration, and 17-20 for interview practice. Speak in your own words; do not claim that planned production deployment has already happened.

Contents and study path

- 1. Project in one minute; 2. Problem and users; 3. What the assignment requires; 4. Architecture; 5. Why these technologies
- 6. Database and ER model; 7. Keys and normalization; 8. Main user journeys; 9. CRUD and website sync; 10. ATS scoring
- 11. Resume, Gemini, and Supabase; 12. Security and deployment; 13. Every project file; 14. Setup and commands; 15. Demo script
- 16. Presentation speaking notes; 17. Beginner questions; 18. DBMS and backend questions; 19. AI, security, and advanced questions; 20. Final revision sheet

Terms to know before presenting

Term	Meaning in this project
Frontend	HTML/CSS/JavaScript pages the user sees in a browser.
Backend	Flask routes that validate input, apply rules, and query MySQL.
Database	MySQL tables that retain users, jobs, applications, skills, and analyses.
ORM	SQLAlchemy maps Python classes to SQL tables and rows.
CRUD	Create, Read, Update, Delete: the four basic data actions.
ATS	Here, a transparent required-skill match score for a job application.
API	A request/response interface; the server calls Google's Gemini API.

1. Project in one minute

DevMatch AI is a web application for software hiring. Developers build a profile, list skills and projects, upload a PDF resume, search jobs, and apply. Recruiters create jobs, review applicants, inspect matched and missing skills, and change application statuses. The application stores relational records in MySQL. A separate DBMS demo page shows live job rows so SQL changes can be seen in both PowerShell and the browser. Gemini can analyze resume text and answer career questions when an API key is configured.

Interview-ready summary: 'I built a two-role recruitment website using Flask, SQLAlchemy, and MySQL. I normalized skills and job requirements into related tables, added role-based CRUD, and verified that database changes appear in the browser. I implemented a transparent required-skill match rule for initial application status and added optional Gemini-based resume analysis and career advice. MySQL remains the source of truth; Supabase is only an optional private file store.'

2. Problem, users, and scope

The problem is that candidate information, job requirements, and application decisions are scattered. A relational system can connect candidates, recruiters, postings, skills, and decisions in a way that can be queried and demonstrated. The project is a DBMS mini project, so its schema and SQL behavior matter as much as its interface.

- Developer role: register/login, maintain profile, add/update/delete skills, add/delete projects, upload resume, browse jobs, apply once per job, see match and status, ask Gemini for career advice.

- Recruiter role: register/login, maintain company profile, create/edit/delete jobs, browse developers, review applications and skill evidence, override application status.
- Public visitor: view landing page, browse jobs, and inspect the read-only DBMS demo table. Authentication is required for profile and hiring actions.

The application is designed for a desktop/laptop browser, matching the guideline. It is responsive, but it is not a native mobile app. This is a demonstration system, not a validated hiring product.

3. Assignment guidelines and evidence

The source PowerPoint requires at least five tables and at least five rows per table, MySQL commands including INSERT/DELETE/UPDATE/MODIFY/ALTER/VIEW/TRIGGER, a desktop/laptop application, schema and ER diagrams, ER mapping, normalization, and a development process. The guideline allows Python.

Requirement	Where this project shows it
5+ related tables	10 SQLAlchemy application tables in app.py, plus application_status_history created by mysql_demo.sql.
5+ rows each	seed-demo loads samples; verify-dbms counts rows and attributes. Current local check passed.
MySQL commands	mysql_demo.sql contains DDL/DML, a view, trigger, ALTER/MODIFY, joins, grouping, and transaction example.
ER/schema/normalization	DBMS_PROJECT_DOCUMENTATION.md contains relational schema, ER diagram, mapping rules, and 1NF-3NF rationale.
Frontend/backend link	Job SQL changes show on /dbms-demo after refresh; recruiter forms write to the same MySQL database.

Observed local database counts on 29 Sep 2026: users 12, developer_profiles 6, recruiter_profiles 6, skills 15, projects 5, project_technologies 10, jobs 15, job_skills 27, applications 10, resume_analysis 6, application_status_history 7. Counts can change as you use CRUD.

4. Architecture: what talks to what

```
Browser (HTML/CSS/JavaScript)
       | HTTP GET/POST
       v
Flask routes in app.py -> validation, login, ATS, page rendering
       | SQLAlchemy + PyMySQL
       v
MySQL 8 -> users, profiles, jobs, skills, applications, analysis
       |
       + optional: local uploads or private Supabase PDF bucket
       + optional: Gemini API for resume/advisor text
```

The browser never connects directly to MySQL or receives the Gemini key. A GET request reads records and renders a Jinja template. A POST request checks the user and CSRF token, validates form data, writes via SQLAlchemy, commits, then redirects. When a page reloads, its next database query sees committed changes.

The DBMS page at **/dbms-demo** performs a fresh Job query. It has a title filter and disables browser caching. It is read-only and shows public job information: ID, title, company, location, and type. It makes the SQL-to-website demonstration easy to see.

5. Technology choices and alternatives

Choice	Why used here	Alternative and tradeoff
Python + Flask	Small, clear server with routes and templates; Python fits the guideline and AI/PDF libraries.	Django has more built-in admin and structure but more framework concepts; Express requires JavaScript backend and separate ORM choices.
MySQL 8	Required MySQL commands; relational keys, constraints, joins, views, and triggers are central to the rubric.	SQLite is simple locally but lacks the same MySQL-specific demonstration; Supabase PostgreSQL would conflict with the selected MySQL requirement.
SQLAlchemy	Models and relationships keep Python CRUD readable and use parameterized queries.	Raw SQL exposes every statement but adds repetition and more injection risk if used badly; Django ORM is tied to Django.
Jinja + HTML/CSS/JS	Server-rendered pages are easy to inspect and run without a separate frontend build.	React can provide richer client state but requires another app, API contract, and build pipeline.
PyMySQL	Connects SQLAlchemy to MySQL from Python.	mysqlclient may be faster but can be harder to install on a classroom Windows setup.
PyMuPDF	Extracts selectable text from uploaded PDF resumes.	OCR is needed for scanned image PDFs; this project currently rejects textless PDFs.
Gemini API	Optional structured resume text analysis and career guidance.	Local models need more compute; another hosted AI provider would change cost, API, and privacy handling.
Supabase Storage	Optional private PDF bucket for production-style file storage.	Local files are simpler for a demo but disappear on ephemeral hosting; Supabase PostgreSQL is deliberately not used.
Git/GitHub	Tracks source changes and allows sharing.	A ZIP has no change history or collaboration workflow.

These are design choices for this assignment, not a claim that alternatives are inherently worse. For an interview, connect each choice to a requirement, a practical constraint, and its cost.

6. Relational database: tables and responsibilities

Table	Key facts and role
users	Account identity: name, unique email, hashed password, role, time. PK id.
developer_profiles	One developer profile per user; unique FK user_id. Bio, education, experience, links, resume path.
recruiter_profiles	One recruiter profile per user; unique FK user_id. Company details.
skills	A developer's individual skills, level, and years. FK developer_id; unique (developer_id, name).
projects	Portfolio projects owned by a developer. FK developer_id.
project_technologies	Individual project technologies. FK project_id; unique (project_id, name).
jobs	Recruiter postings. FK recruiter_id; title, description, location, pay text, type, deadline, time.
job_skills	Individual required skills for a job. FK job_id; unique (job_id, name).
applications	Developer-to-job link with status and time. FKs to both; unique (developer_id, job_id).
resume_analysis	Each uploaded analysis stores summary and extracted skills for a developer. FK developer_id.
application_status_history	SQL-created audit table. FK application_id; old/new status and changed time.

Important distinction: the first ten are SQLAlchemy models. The history table is created by the MySQL demo script, not by a Python model. The script also creates the view **job_application_summary**, which is a saved query, not a normal stored table.

7. Relationships, keys, and normalization

ER model in words

```
User 1 -- 0..1 Developer -- many Skills / Projects / Analyses
User 1 -- 0..1 Recruiter -- many Jobs -- many JobSkills
Developer many -- Applications -- many Jobs
Application 1 -- many StatusHistory records
Project 1 -- many ProjectTechnologies
```

A primary key identifies a row. A foreign key points to another table. Unique constraints enforce business rules: one profile per account, no duplicate developer skill name, no duplicate job skill name, and no second application by the same developer to the same job. Cascades on ORM relationships remove dependent records when parents are deleted through the application; understand that direct SQL deletes still depend on database FK behavior.

Many-to-many example: a developer can apply to many jobs and a job can receive many developers. The **applications** table resolves this with two foreign keys and adds facts belonging to the relationship: status and application time.

Normalization example

An early-style job record could store 'Python, Flask, MySQL' in one required_skills text field. That makes searching, joining, and enforcing one skill per row difficult. The normalized design stores one job row and three job_skills rows. Likewise, project technologies are child rows. The init-db migration moves old comma-separated values into child tables and drops the old columns.

- 1NF: each column value is atomic for the modeled relation; repeatable skills are rows, not a comma-separated list in jobs.
- 2NF: facts about a job-skill pair, such as importance, belong to that pair, not only the job or skill name.
- 3NF: recruiter company facts live in recruiter_profiles rather than being copied into every job; user credentials stay in users rather than each profile.

Interview nuance: resume_analysis currently stores extracted skills as comma-separated text, so it is less normalized than the core job/project skill design. It is a derived snapshot rather than a canonical skill catalog. If expanding the system, use a separate resume_analysis_skills table.

8. Main workflows from screen to database

Registration and login

The registration form creates a users row plus either a developer_profiles or recruiter_profiles row in one SQLAlchemy session. Passwords are stored as PBKDF2-SHA256 hashes, not plaintext. Login verifies the hash and stores the user ID in a Flask session cookie. login_required enforces sign-in and optional role checks.

Recruiter job CRUD

A recruiter submits a job form with title, description, and required skills. create_job builds one jobs row and child job_skills rows, then commits. The edit route checks that the job belongs to that recruiter, changes fields, replaces its required-skill child rows, and commits. Delete checks ownership and removes the job through ORM. The public jobs page and DBMS demo page read the updated database.

Developer application

The developer clicks Apply. The route checks job existence and deadline, prevents repeat applications, calculates required-skill match, inserts an application with initial Shortlisted or Rejected status, commits, and shows feedback. A recruiter can later override the status. A MySQL trigger records status changes in application_status_history when installed.

Developer profile and resume

Profile/skill/project forms update related tables. Resume upload checks PDF extension, MIME type, PDF signature, size, and selectable text. PyMuPDF extracts text. The file goes to local uploads in the current local configuration or to a private Supabase bucket when configured. The analysis result becomes a resume_analysis row. Recruiters can view candidate evidence through authorized routes.

9. CRUD and why the website changes

Letter	SQL example	Visible outcome
Create	INSERT INTO jobs (...) VALUES (...);	A new job row appears after page refresh.
Read	SELECT id, title, location FROM jobs;	The DBMS page displays the same saved values.
Update	UPDATE jobs SET location='Delhi' WHERE id=...;	The location cell changes after refresh.
Delete	DELETE FROM jobs WHERE id=...;	The row disappears after refresh.

The website is not copying PowerShell output into HTML. Both PowerShell's MySQL client and Flask connect to the same MySQL database. The SQL command commits a change. A new GET /dbms-demo request queries jobs again and renders fresh HTML. The no-store response header discourages cached results. There is no live WebSocket push: click Refresh table or reload the page.

The companion PDF **DevMatch_CRUD_PowerShell_and_Website_Guide.pdf** gives exact VS Code PowerShell commands. mysql_shell.py reads the local .env and launches mysql.exe so you can use --execute without showing the password in a command.

Demonstration SQL ideas

```
SELECT COUNT(*) FROM jobs;
SELECT j.title, r.company_name FROM jobs j
JOIN recruiter_profiles r ON r.id = j.recruiter_id;
SELECT status, COUNT(*) FROM applications GROUP BY status;
SELECT * FROM job_application_summary;
SHOW TRIGGERS;
```

A JOIN combines related tables using keys. GROUP BY summarizes rows. A VIEW saves a query definition. A TRIGGER reacts to a database event. ALTER TABLE ... MODIFY changes a column definition. The SQL script also uses a transaction with ROLLBACK to demonstrate write commands without keeping its test application.

10. ATS required-skill score and status rule

For a job, make a case-insensitive set of required job_skills. Make another set from the developer's profile skills and the latest resume analysis's extracted skills. The matched set is the intersection of required skills with the union of those candidate sets. Missing skills are required minus matched.

```
score shown = round(100 * matched_required / total_required)
shortlist = total_required > 0 AND
            100 * matched_required > 70 * total_required
else initial status = Rejected
```

Example: a job requires Python, Flask, MySQL, SQL. A candidate has Python, Flask, MySQL: 3 of 4 = 75%, so initial status is Shortlisted. A candidate with only Python and Flask scores 50%, so initial status is Rejected. Exactly 70% does not pass because the condition is **greater than** 70, not greater than or equal to 70.

The code compares integer counts before rounding; this avoids a rounding error in the rule. An edge case is that a raw score slightly above 70% can display as 70% after rounding while being shortlisted. For a production interface, display one decimal place or the matched count to explain this. Jobs inserted directly with no job_skills yield a 0% match and an initial Rejected status, so create child skills for realistic ATS demonstrations.

This is a simple skills-match rule, not a general commercial ATS score and not proof of applicant suitability. It does not verify proficiency, recency, experience quality, or fairness. A recruiter can review evidence and change the status. Explain those limits honestly in an interview.

11. Gemini, resume analysis, and file storage

What Gemini does and does not do

The server sends extracted resume text to Gemini with a requested JSON shape: summary, strengths, suggested development areas, and explicitly mentioned skills. It validates the returned structure before saving. If the key is missing or the call fails, a local keyword extractor returns a basic analysis so resume upload can still work. The local system has a Gemini key configured, but it is kept in the ignored .env file.

The developer-only /career-ai page sends a limited profile summary, skills, up to five recent roles, and the user's question to Gemini. It returns practical text. Gemini is **not** the source of the deterministic >70% application rule. The career page explains that profile details and the question are sent to Google when submitted.

The current local configuration uses local resume files, not Supabase Storage. Production configuration expects a private Supabase bucket, a server-side service-role key, and MySQL for relational data. Do not say that Supabase PostgreSQL stores application tables. Do not show the actual Gemini key, database password, or service-role key in a

presentation.

A useful interview explanation

'I separate deterministic application status from generative advice. The skill match can be reproduced from table rows. Gemini analyzes free text and gives optional guidance, but API outputs can be incomplete or wrong, so the system validates structure and has a local fallback.'

12. Security, deployment, and honest limits

- Passwords are hashed; sessions keep only a user ID. Routes enforce developer/recruiter roles and ownership checks for job edits and skill updates.
- POST forms include a CSRF token. Login and registration are rate limited. PDFs are size limited and checked for type, signature, and extractable text.
- .env, local database files, resume uploads, logs, virtual environments, MySQL binaries, and temporary previews are excluded from Git. Examples use placeholders.
- Production configuration requires allowed hosts, a real secret, Redis-based limiter storage, Supabase Storage, and a reachable MySQL server. render.yaml describes a Render service; that is deployment configuration, not evidence of a live deployment.

Limitations to say clearly: The ATS rule is simplistic and should not be a final hiring decision. Gemini suggestions can be inaccurate. Scanned PDFs need OCR. Current local Supabase storage is not configured. Demo accounts use a shared sample password and should not be publicly deployed. No automated test suite is present; the database verifier and manual route/CRUD checks cover the class demonstration. The application_status_history audit table is created by mysql_demo.sql, not by init-db alone.

Next technical improvements: add proper migration tooling, automate route and database tests, use OCR for scanned resumes, store resume skills in normalized rows, review privacy and retention requirements, and align displayed ATS precision with the exact threshold comparison.

13. File-by-file map: the project folder

The main source folder is **DevMatch_AI_DBMS_Mini_Project/devmatch-ai**. These are the committed source files and their actual roles. Files marked legacy are present but are not rendered by current Flask routes.

File	What it does
app.py	Main Flask app: settings, models, routes, auth, CSRF, CRUD, ATS, resume/Gemini logic, and CLI commands.
requirements.txt	Python dependency list installed into .venv.
README.md	Setup, feature overview, local MySQL, demo, and deployment instructions.
DBMS_PROJECT_DOCUMENTATION.md	ER/schema documentation, mapping, normalization, rubric and SQL explanation.
mysql_demo.sql	MySQL view, trigger, history table, ALTER/MODIFY, joins, grouping, and transaction demonstration.
mysql_shell.py	Starts mysql.exe with host, port, user, and database read from local .env; supports --execute.
render.yaml	Blueprint-style deployment configuration; not needed to run locally.

File	What it does
.env.example	Safe local configuration template with placeholders.
.env.production.example	Safe production variable template; no actual credentials.
.gitignore	Excludes local .env, uploads, logs, caches, virtualenv, and generated DB file from Git.

templates/ - page layouts

Template	Purpose
base.html	Shared navigation, CSS links, footer, flash messages, theme and UI JavaScript.
career_home.html	Current public landing page for signed-out visitors.
auth.html	Login and registration forms.
developer.html	Developer dashboard: profile context, skills/projects, resume, applications, suggestions.
recruiter_dashboard.html	Recruiter job CRUD, applicant evidence and status controls.
jobs.html	Public searchable job cards and Apply buttons for developers.
dbms_demo.html	Read-only MySQL jobs table, count, search, and refresh control.
edit_job.html	Recruiter job editing form.
settings.html	Profile and password settings.
developers.html	Recruiter search results for developer profiles.
developer_detail.html	Recruiter view of a candidate profile and analysis.
career_ai.html	Gemini career question form and answer.
index.html	Older landing template; no current route renders it.
recruiter.html	Older recruiter template; current route uses recruiter_dashboard.html.

static/ - browser styling

File	Purpose
style.css	Base layout, forms, dashboard, cards, and common component styles.
career.css	Additional career-site layout and responsive styling.
career_ai.css	Gemini advisor-specific styles.
dbms_demo.css	DBMS demo table, summary, search, and responsive layout.

Local/generated folders and the repository root

.env holds local secrets and is ignored. **.venv/** holds installed Python packages. **uploads/** stores local resume PDFs; only uploads/.gitkeep is committed so the empty folder exists. **__pycache__/**, server logs, devmatch.db, and instance/ are generated or legacy local artifacts, not the active MySQL database. At the repository root, the original guidelines PPTX is the assignment source; **output/pdf/** holds generated handouts; **mysql-local/** and **mysql-8.4.11-winx64/** are local MySQL runtime/data, excluded from Git. The Git repository contains source, not the running database rows or secrets.

14. Run, inspect, and debug

Start in VS Code PowerShell

```
cd D:\DBMS_DevApp\DevMatch_AI_DBMS_Mini_Project\devmatch-ai
& .\.venv\Scripts\python.exe -m flask --app app verify-dbms
& .\.venv\Scripts\python.exe -m flask --app app run --debug --port 5001
```

Open **http://127.0.0.1:5001** for the site and **http://127.0.0.1:5001/dbms-demo** for the live table. If port 5001 is occupied, the server may already be running. Keep a second terminal for SQL commands.

```
cd D:\DBMS_DevApp\DevMatch_AI_DBMS_Mini_Project\devmatch-ai
& .\.venv\Scripts\python.exe .\mysql_shell.py `
  --execute "SELECT id, title, location FROM jobs LIMIT 5;"
```

For an interactive mysql> prompt, run the helper without --execute. The CLI commands init-db and seed-demo are for creating a fresh demonstration database; do not run seed-demo casually against a database with your own records. verify-dbms is safe to run and reports row/column/relationship checks.

Troubleshooting sequence

- Website does not start: confirm the current directory is devmatch-ai, use that folder's .venv, ensure .env has DATABASE_URL, and check whether MySQL is running on the configured port.
- SQL change does not show: use /dbms-demo, clear the search or use the matching title, click Refresh table, and confirm the SQL command and website use the same database.
- A new job has 0% ATS score: check its job_skills rows; bare SQL INSERT into jobs does not create required skills automatically.
- Gemini response unavailable: check the server-side key, selected model, network/API response, and rate limit. Resume analysis still has the local extractor fallback.
- Git status shows untracked local files: add only intended source or handouts. Do not add .env, uploads, local MySQL data, or resume files.

15. Live demonstration plan: about 8 minutes

Time	Show	Say
0:00-0:45	Landing page and roles	'This is a two-role recruitment system backed by MySQL.'
0:45-1:45	Schema/ER diagram	'Users split into role profiles; applications join developers to jobs.'
1:45-2:30	verify-dbms output	'Ten related application tables satisfy row and attribute minimums.'
2:30-4:30	PowerShell CRUD + /dbms-demo	'Both tools read the same MySQL database; refresh shows each committed change.'
4:30-5:30	Recruiter dashboard	'A recruiter owns job CRUD and can review applicant evidence.'
5:30-6:20	ATS example	'3 of 4 required skills is 75%, so the initial status is Shortlisted; exactly 70% fails.'
6:20-7:10	Resume/Gemini page	'Gemini is optional for text analysis/advice. It does not decide ATS status.'
7:10-8:00	View and trigger SQL	'The view summarizes applications and trigger records status changes.'

Before presenting, keep the website and MySQL running, rehearse the specific account logins, and use a clearly named disposable CRUD row. Have the ER diagram and this guide open. Do not expose .env or real uploaded resumes on screen.

16. Presentation speaking notes

Opening - around 60 seconds

'My project is DevMatch AI, a web-based recruitment and developer portfolio application. The main objective was to apply relational database design to a real workflow. Developers store profiles, skills, projects, resumes, and applications. Recruiters publish jobs and review candidates. The database is MySQL; Flask handles routes and forms; SQLAlchemy maps our Python models to related tables. I also added optional Gemini assistance, while the application-status rule remains a transparent skills calculation.'

Database explanation - around 90 seconds

'The schema has ten application tables. The users table holds login identities; developer_profiles and recruiter_profiles are one-to-one role details. A recruiter has many jobs. Each job has many job_skills. A developer has many skills and projects. Applications connect developers and jobs and store status. This is the many-to-many bridge. Unique and foreign-key constraints prevent duplicate applications and broken references. Skills and project technologies are child rows, so we avoid comma-separated fields in those core relations.'

CRUD explanation - around 60 seconds

'I first show at least five records. Then I insert a job in MySQL and refresh the DBMS page; it appears. I update the location and refresh; the cell changes. I delete a temporary job and refresh; it disappears. Both PowerShell and the website connect to the same MySQL database, so this proves that the frontend reflects backend data.'

ATS and AI explanation - around 60 seconds

'The ATS-like score compares a job's required skills with the union of the candidate's profile skills and latest extracted resume skills. If more than 70 percent match, the initial status is Shortlisted; otherwise it is Rejected. Recruiters can review and override it. Gemini is separate: it analyzes resume text and gives career advice. Its output is validated and there is a local fallback if the API is unavailable.'

Closing - around 30 seconds

'The project demonstrates relational modeling, normalized child tables, primary and foreign keys, SQL CRUD, JOIN, GROUP BY, a view, a trigger, and the database-to-website link. It also shows how a practical application can use deterministic rules and optional AI while keeping MySQL as the source of truth.'

17. Interview questions: beginner

What is your project in one sentence?

A two-role recruitment website where developers and recruiters work with records stored in MySQL, with a transparent skill match and optional Gemini assistance.

Why did you build it?

It gives a realistic setting for relational design, CRUD, keys, joins, and a visible frontend/backend link required by the DBMS assignment.

Who can use the website?

Public visitors can browse jobs; registered developers maintain profiles and apply; recruiters post jobs and review applications.

What is frontend versus backend?

Frontend is browser HTML/CSS/JavaScript; backend is Flask logic that validates requests, handles sessions, calculates scores, and accesses MySQL.

What is Flask?

A Python web framework that maps HTTP requests to route functions and returns rendered pages or responses.

What is a route?

A URL and HTTP method mapped to a server function, such as GET /jobs or POST /jobs.

What is Jinja?

Flask's template engine; it inserts server-provided values into HTML and escapes output by default.

What is MySQL doing?

It persistently stores the relational application records and enforces keys, constraints, views, and triggers.

What is SQLAlchemy?

An ORM that maps Python classes and relationships to database tables and helps perform CRUD through sessions.

What does CRUD stand for?

Create, Read, Update, Delete - INSERT, SELECT, UPDATE, DELETE at the SQL level.

Why does the website show SQL changes after refresh?

A new GET request queries the same MySQL database after the SQL change is committed, then renders current values.

What is .env?

A local environment configuration file containing values such as DATABASE_URL and GEMINI_API_KEY; it is ignored by Git.

What is a virtual environment?

A project-local Python package installation area, .venv, that avoids mixing dependencies with other projects.

What does Git track?

Source files and their history. It does not automatically track MySQL rows, ignored secrets, or local uploads.

Is the site already deployed?

The repository includes Render configuration, but a deployment should only be claimed after a live URL and production dependencies are verified.

18. Interview questions: DBMS and backend

What is a primary key?

A unique, non-null identifier for one row; each core table has an integer id primary key.

What is a foreign key?

A child column that references a parent key, such as jobs.recruiter_id pointing to recruiter_profiles.id.

Why separate users and profiles?

Shared login facts stay in users; developer and recruiter fields belong to their distinct roles. The profile user_id is unique for a one-to-one link.

What is a one-to-many relation here?

One recruiter can post many jobs; each jobs row stores a recruiter_id foreign key.

Where is the many-to-many relation?

Developers can apply to multiple jobs and jobs can have multiple developers; applications is the bridge.

Why put status in applications?

Status describes one developer's application to one job, not the developer or job alone.

How is duplicate application prevented?

The route checks first, and a database unique constraint on developer_id plus job_id is the stronger final guarantee.

What is normalization?

Organizing tables so repeatable and dependent facts live in the right relation, reducing duplicate data and update anomalies.

Explain 1NF with your project.

Required job skills are separate job_skills rows instead of a comma-separated jobs column.

Explain 2NF with your project.

A fact about a job-skill requirement, such as importance, belongs to that particular pair rather than just the job.

Explain 3NF with your project.

Recruiter company details are stored in recruiter_profiles; jobs reference that profile rather than repeating its company name.

What is a JOIN?

A query that combines related tables by key, such as jobs with recruiter_profiles to show company names.

What is GROUP BY used for?

It groups rows for aggregates, for example count applications by status.

What is a VIEW?

A saved SELECT definition; job_application_summary joins jobs, recruiters, and applications to show counts.

What is a TRIGGER?

Database logic that runs on an event; log_application_status inserts an audit record after an application status update.

What is ALTER TABLE MODIFY?

ALTER changes table structure; MODIFY changes a column definition, such as widening the salary field.

What is a transaction?

A unit of database work that commits together or rolls back. The SQL demo wraps test DML and rolls it back.

What is the difference between COMMIT and ROLLBACK?

COMMIT makes transaction changes permanent; ROLLBACK cancels uncommitted changes.

Why use a unique constraint as well as validation?

Two concurrent requests can pass application-level checks; the database constraint still prevents duplicates.

What does db.session.commit() do?

It flushes the ORM changes and commits the SQL transaction so later requests can see them.

How does job edit ownership work?

The edit/delete queries filter by both job ID and the signed-in recruiter's ID; other recruiters get 404.

What are migrations in this code?

init-db creates tables and moves legacy comma-separated job/project values into child tables before dropping old columns. It is limited compared with a full migration framework.

19. Interview questions: ATS, AI, security, and advanced

How exactly is ATS score calculated?

Required skills are a case-insensitive set. Candidate skills are the union of profile skills and latest extracted resume skills. Score is matched-required count divided by total-required count, shown as a rounded percentage.

Does 70% get shortlisted?

No. The rule is strictly greater than 70%; exactly 70% gets initial Rejected status.

Why compare counts before rounding?

Rounding can move a raw percentage across a threshold. The status rule uses integer cross multiplication to preserve the strict comparison.

What if a job has no required skills?

The score is 0 and should_shortlist is false. SQL-only demo jobs need job_skills rows for meaningful ATS behavior.

Does Gemini choose who gets shortlisted?

No. Gemini optionally extracts resume information and gives advice. The application-status decision uses a deterministic skill-set rule.

What happens if Gemini fails?

Resume analysis falls back to local keyword extraction. The advisor shows an error and asks the user to retry.

Why validate Gemini JSON?

A model may return malformed or unexpected text. The code checks required shapes and limits lengths before saving analysis.

What is prompt injection in this context?

A resume or question could contain instructions aimed at the model. Prompts tell Gemini to treat resume and context as data; stronger production defenses would be needed.

What data goes to Gemini?

For resume analysis, extracted PDF text. For career advice, the question and limited developer profile/job context. The API key remains server-side.

Why use Supabase if MySQL is the DB?

The optional Supabase component stores private PDF files. MySQL stores relational application rows, meeting the DBMS requirement.

What is a password hash?

A one-way stored verifier. The app uses Werkzeug PBKDF2-SHA256 and checks submitted passwords against the hash.

What is CSRF?

Cross-site request forgery: another site could try to submit a form using an existing session. The app checks a session token on state-changing form requests.

What is SQL injection and how is it limited?

Injection is attacker-controlled text changing query meaning. ORM filters and parameterized SQL keep values separate from SQL structure; never concatenate untrusted input into raw SQL.

Why rate-limit login?

It slows repeated password attempts. Local mode uses memory storage; production requires shared Redis storage.

What happens when a parent row is deleted?

ORM relationships specify delete-orphan cascades for many children. Direct SQL behavior depends on FK constraints and should be verified before deleting parents.

What is the N+1 query problem?

Rendering a list can issue one extra query per row when related objects are lazily loaded. The current templates may benefit from joinedload/selectinload if dataset size grows.

Would this scale to thousands of jobs?

The DBMS demo currently loads all matching jobs and the jobs page all results. Add pagination, indexes for search, and query profiling before large-scale use.

What are the limits of LIKE search?

Percent-wildcard LIKE can require scans and offers limited relevance ranking. Full-text search or a dedicated search engine would help at scale.

How would you make the ATS rule fairer?

Avoid automatic final rejection, validate skills evidence and outcome quality, audit disparate impact, expose reasons, permit review/appeal, and involve hiring experts.

How would you test this application?

Unit-test score boundaries and validation; integration-test MySQL constraints and trigger; route-test role access, CSRF, and CRUD; browser-test visible updates.

What would you change first for production?

Add full migrations and automated tests, security/privacy review, file scanning, retention policy, monitored hosted MySQL, and stronger hiring oversight.

What is the difference between a demo and production deployment?

A demo proves features locally. Production needs managed services, secrets, backups, monitoring, TLS, privacy terms, migrations, and operational review.

How do you explain your contribution honestly?

Describe which parts you personally designed, integrated, tested, and learned. Be clear that you used an AI coding assistant if asked, and explain the code and tradeoffs yourself.

20. Final revision sheet

Remember	Exact answer
Purpose	Developer portfolio + recruiter hiring workflow as a relational DBMS application.
Stack	Python, Flask, SQLAlchemy, PyMySQL, MySQL, Jinja, HTML/CSS/JS, PyMuPDF; optional Gemini and Supabase Storage.
Database	10 ORM tables + SQL-created status-history table; keys, unique constraints, child tables, view, trigger.
ATS	Matched required skills / total required skills; strictly greater than 70% shortlists; recruiter may override.
CRUD proof	SQL in PowerShell -> same MySQL -> refreshed /dbms-demo table changes.
Gemini	Optional resume text analysis and developer career advice; no direct hiring status decision.
Honest limitation	Simple ATS, no OCR, no automated test suite, local Supabase storage not configured, deployment unverified.

Last rehearsal: explain the architecture without notes; draw the ER relationships from memory; calculate one ATS example; execute one CREATE/READ/UPDATE/DELETE sequence; describe one design tradeoff and one limitation. If you can do those five tasks, you can handle most project viva questions.